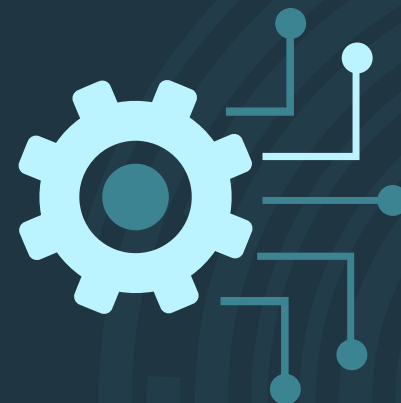




CPU PROCESS SCHEDULING

ระบบจัดคิว PROCESS ของ CPU



1. PROBLEM ANALYSIS

1.1 PROBLEM

ระบบปฏิบัติการต้องจัดสรร CPU ให้กับ Process หลายตัวที่ ร้องการประมวลผล โดยแต่ละ Process มีเวลาที่ ต้องใช้ CPU แตกต่างกันไป จึงต้องมี Algorithm สำหรับกำหนดลำดับ การใช้ CPU รายงานนี้ศึกษาการจัดคิว Process โดยเปรียบเทียบ First Come First Served (FCFS) และ Round Robin (RR)

Algorithm A: First Come First Served (FCFS) จัด Process ตามลำดับที่ เข้ามาใน Queue โดย Process ที่เข้ามาก่อนจะได้รับ CPU ก่อน

Algorithm B: Round Robin (RR) แบ่งเวลา CPU ให้แต่ละ Process ตาม ลำดับ โดยกำหนด Time Quantum = 3 หาก Process ยังทำงานไม่เสร็จหลัง ใช้ CPU ครบ 3 หน่วยเวลา จะถูกนำกลับไปต่อท้าย Circular Queue



1.2 INPUT

Process	Arrival Time	Burst Time
P1	0	8
P2	0	4
P3	0	9
P4	0	5

Time Quantum ของ Round Robin = 3

1.3 OUTPUT

- ลำดับการทำงานของ Process
- Waiting Time ของแต่ละ Process
- Turnaround Time ของแต่ละ Process
- จำนวน Context Switch
- Average Waiting Time
- Average Turnaround Time
- ผลการเปรียบเทียบ FCFS และ Round Robin





1.4 CONSTRAINTS

- มี Process ทั้งหมด 4 Process
- กำหนด Arrival Time = 0 สำหรับทุก Process
- Burst Time ของแต่ละ Process แตกต่างกันไป
- FCFS ทำงานตามลำดับ FIFO
- Round Robin ใช้ Circular Queue
- Time Quantum ของ Round Robin เท่ากับ 3
- Process ที่ยังทำงานไม่เสร็จจะถูกนำกลับไปต่อท้าย Queue



1.5 QUEUE เก็บข้อมูลอะไร

Queue เก็บข้อมูลของ Process ที่รอใช้ CPU ได้แก่ Process ID, Arrival Time, Burst Time และ Remaining Time

2. QUEUE DESIGN

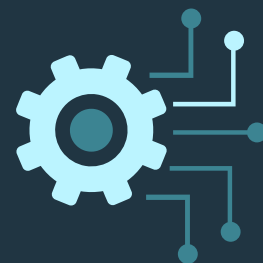


2.1 ALGORITHM A: FCFS

- ใช้ Queue แบบ FIFO (First In First Out) ลำดับเริ่มต้น: $\text{Front} \rightarrow P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow \text{Rear}$
ลำดับการทำงาน: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$

2.2 ALGORITHM B: ROUND ROBIN

- ใช้ Circular Queue และกำหนด Time Quantum = 3 เมื่อ Process ได้ CPU จะทำงานได้สูงสุด 3 หน่วยเวลา ถ้าทำงานเสร็จจะถูกนำออกจาก Queue แต่ถ้ายังไม่เสร็จจะถูกนำกลับไปต่อท้าย Queue





3. PSEUDOCODE ALGORITHM A — FCFS

อัลกอริทึม FCFS (First Come First Served)

1. สร้างคิวว่าง (Queue)
 2. นำ P1, P2, P3 และ P4 เข้าไปในคิว
 3. กำหนด $\text{currentTime} = 0$
 4. ทำซ้ำขณะที่คิวไม่ว่าง
 - 4.1 นำ Process ที่อยู่ด้านหน้าสุดของคิวออก (Dequeue)
 - 4.2 ประมวลผล Process นั้นจนทำงานเสร็จ
 - 4.3 คำนวณเวลาที่ Process ทำงานเสร็จ (Completion Time)
 - 4.4 คำนวณเวลารอคอย (Waiting Time)
 - 4.5 คำนวณเวลาตั้งแต่เข้าคิวจนทำงานเสร็จ (Turnaround Time)
 5. คำนวณค่าเฉลี่ยเวลารอคอย (Average Waiting Time)
 6. คำนวณค่าเฉลี่ยเวลาตั้งแต่เข้าคิวจนทำงานเสร็จ (Average Turnaround Time)
 7. นับจำนวนการเปลี่ยน Process (Context Switch)
 8. แสดงผลลัพธ์
- 



4. PSEUDOCODE ALGORITHM B — ROUND ROBIN

อัลกอริทึม Round Robin

1. สร้างคิวแบบวงกลม (Circular Queue) ว่าง
2. กำหนด Time Quantum = 3
3. นำ P1, P2, P3 และ P4 เข้าไปในคิว
4. กำหนด currentTime = 0
5. ทำซ้ำขณะที่คิวแบบวงกลมไม่ว่าง
 - 5.1 นำ Process ที่อยู่ด้านหน้าสุดของคิวออก (Dequeue)
 - 5.2 ประมวลผล Process เป็นเวลา $\min(\text{Time Quantum}, \text{Remaining Time})$
 - 5.3 ปรับปรุงค่า currentTime
 - 5.4 ปรับปรุงเวลาที่เหลือในการทำงาน (Remaining Time)
 - 5.5 ถ้า Remaining Time = 0
 - ถือว่า Process ทำงานเสร็จ
 - คำนวณเวลาที่ Process ทำงานเสร็จ (Completion Time) ถ้า Remaining Time > 0
 - นำ Process กลับเข้าไปต่อท้ายคิว (Enqueue)
6. คำนวณเวลารอคอย (Waiting Time)
7. คำนวณเวลาตั้งแต่เข้าคิวจนทำงานเสร็จ (Turnaround Time)
8. นับจำนวนการเปลี่ยน Process (Context Switch)
9. คำนวณค่าผลลัพธ์เฉลี่ย
10. แสดงผลลัพธ์



5. QUEUE TRACE

Round Robin, Time Quantum = 3

Step	Operation	Queue Before	CPU	Remaining หลังทำงาน	Queue After
1	ENQUEUE P1	[]	-	P1=8	[P1]
2	ENQUEUE P2	[P1]	-	P2=4	[P1,P2]
3	ENQUEUE P3	[P1,P2]	-	P3=9	[P1,P2,P3]
4	ENQUEUE P4	[P1,P2,P3]	-	P4=5	[P1,P2,P3,P4]
5	DEQUEUE P1	[P1,P2,P3,P4]	P1 ใช้ 3	P1=5	[P2,P3,P4,P1]
6	DEQUEUE P2	[P2,P3,P4,P1]	P2 ใช้ 3	P2=1	[P3,P4,P1,P2]
7	DEQUEUE P3	[P3,P4,P1,P2]	P3 ใช้ 3	P3=6	[P4,P1,P2,P3]
8	DEQUEUE P4	[P4,P1,P2,P3]	P4 ใช้ 3	P4=2	[P1,P2,P3,P4]
9	DEQUEUE P1	[P1,P2,P3,P4]	P1 ใช้ 3	P1=2	[P2,P3,P4,P1]
10	DEQUEUE P2	[P2,P3,P4,P1]	P2 ใช้ 1	P2=0	[P3,P4,P1]
11	DEQUEUE P3	[P3,P4,P1]	P3 ใช้ 3	P3=3	[P4,P1,P3]
12	DEQUEUE P4	[P4,P1,P3]	P4 ใช้ 2	P4=0	[P1,P3]
13	DEQUEUE P1	[P1,P3]	P1 ใช้ 2	P1=0	[P3]
14	DEQUEUE P3	[P3]	P3 ใช้ 3	P3=0	[]

ลำดับ CPU: P1 → P2 → P3 → P4 → P1 → P2 → P3 → P4 → P1 → P3



6. CORRECTNESS / INVARIANT

6.1 FCFS

Algorithm FCFS ถูกต้องเนื่องจากใช้หลัก FIFO ซึ่ง Process ที่เข้าคิวก่อนจะถูกนำออกจาก Queue ก่อน ทำให้ลำดับการประมวลผลเป็นไปตามลำดับการเข้าคิว Invariant : ก่อนการประมวลผลแต่ละรอบ Process ที่อยู่ด้านหน้า Queue คือ Process ที่เข้ามาก่อน Process อื่นทั้งหมดที่ยังไม่ได้รับการประมวลผล

6.2 ROUND ROBIN

Round Robin ถูกต้องเนื่องจากทุก Process จะได้รับโอกาสใช้ CPU ตามลำดับ และ Process ที่ยังไม่เสร็จ หลังครบ Time Quantum จะถูกนำกลับไปท้าย Queue Invariant: ก่อนการประมวลผลแต่ละรอบ Process ที่อยู่ด้านหน้า Circular Queue จะเป็น Process ที่รอการประมวลผลในรอบปัจจุบัน และ Process ที่ยังไม่เสร็จจะถูกนำกลับไปท้าย Queue

7. TIME COMPLEXITY

Operation	FCFS	Round Robin
enqueue()	$O(1)$	$O(1)$
dequeue()	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$
search()	$O(n)$	$O(n)$
display()	$O(n)$	$O(n)$

สำหรับการจัดการ Queue การ enqueue และ dequeue สามารถทำได้ใน $O(1)$ ส่วน search และ display ต้องตรวจสอบสมาชิกของ Queue จึงเป็น $O(n)$ สำหรับ Round Robin จำนวนการทำงานขึ้นกับจำนวน Time Slice โดยเมื่อ Time Quantum เป็นค่าคงที่จำนวนรอบจะแปรผันตามปริมาณ Burst Time ที่ต้องประมวลผล

8. SPACE COMPLEXITY

ทั้ง FCFS และ Round Robin ต้องเก็บ Process ที่รออยู่ใน Queue หากมี Process สูงสุด n รายการ Space Complexity โดยทั่วไปเป็น $O(n)$ โดย Round Robin ยังต้องเก็บ Remaining Time ของแต่ละ Process

9. TEST CASES

Test Case	Input	Expected Result
1. Normal Case	P1=8, P2=4, P3=9, P4=5	Process ทั้งหมดทำงานเสร็จ
2. Empty Queue	ไม่มี Process	ระบบแจ้งว่า Queue ว่าง
3. Single Item	P1=8	P1 ทำงานจนเสร็จ
4. Large Queue	Process จำนวนมาก	ระบบสามารถประมวลผล Process ได้ทั้งหมด
5. Special / Edge Case	Burst Time เท่ากันหลาย Process	Process ทำงานตามกฎของ Algorithm
6. Cancel Case	ยกเลิก Process ที่อยู่ใน Queue	Process ที่ถูกยกเลิกจะไม่ถูกประมวลผล

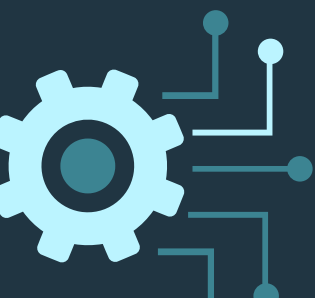


10. EXPERIMENTAL COMPARISON

การทดลองต้องใช้ขนาดข้อมูล $n = 100, 1,000, 10,000$ และ $50,000$ วัด Execution Time ด้วย `nanoTime()` มี warm-up ใช้ seed คงที่และเฉลี่ยผลจาก 5 รอบ

n	FCFS Average Time (ns)	Round Robin Average Time (ns)
100	30,220.00	67,080.00
1,000	120,660.00	410,360.00
10,000	516,020.00	701,480.00
50,000	744,560.00	1,944,600.00

จากผลการทดลองพบว่า เมื่อจำนวน Process เพิ่มขึ้น เวลาที่ใช้ในการประมวลผลของทั้ง FCFS และ Round Robin มี แนวโน้มเพิ่มขึ้น โดยจากการทดลองชุดนี้ FCFS ใช้เวลาในการประมวลผลน้อยกว่า Round Robin ในทุกขนาดข้อมูลที่ ทดสอบ เนื่องจาก FCFS ประมวลผลแต่ละ Process จนเสร็จแล้วจึงไปยัง Process ถัดไป ขณะที่ Round Robin มีการ แบ่ง CPU ตาม Time Quantum = 3 และนำ Process ที่ยังทำงานไม่เสร็จกลับเข้าสู่ Queue ทำให้มีการดำเนินการกับ Queue หลายครั้งกว่า



11. สรุปเปรียบเทียบ ALGORITHM A และ B

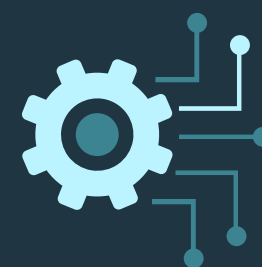
ประเด็น	FCFS	Round Robin
Data Structure	Queue	Circular Queue
หลักการ	มาก่อนได้ก่อน	แบ่งเวลา CPU ตาม Time Quantum
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
Average Waiting Time	10.25	15.00
Average Turnaround Time	16.75	21.50
Context Switch	3	9
Fairness	น้อยกว่าเมื่อมีงานแรกใช้เวลานาน	สูงกว่าเพราะแบ่งเวลาให้แต่ละ Process
ข้อดี	ง่ายและมี Context Switch น้อย	เหมาะกับการให้หลาย Process ได้รับ CPU อย่างทั่วถึง
ข้อจำกัด	Process ที่อยู่หลังงานยาวอาจรอนาน	มี Context Switch มากกว่า
เหมาะกับกรณี	Batch Processing	Interactive System

จากชุดข้อมูลที่กำหนด FCFS มี Average Waiting Time และ Average Turnaround Time ต่ำกว่า Round Robin แต่ Round Robin มีความยุติธรรมในการแบ่งเวลา CPU มากกว่า และเหมาะกับ Interactive System ที่ต้องการให้ผู้ใช้ได้รับการตอบสนองอย่างต่อเนื่อง



GROUP MEMBERS

- | | |
|----------------------------------|-------------|
| 1. นางสาวปริญญิตย์ สิริรัตนติกุล | 67122250046 |
| 2. นางสาวกัญญวรา พุนช่วย | 68122250011 |
| 3. นางสาวศิราณี เสาคง | 68122250036 |
| 4. นายชลลสิทธิ์ งามเงินทวี | 68122250096 |
| 5. นายปิยวัฒน์ มีที | 68122250101 |



THANK
YOU

